

18주 차 보고서

(2026년 4 월 26일 ~ 2026년 5월 2 일)

작성자

박신우

이번 주 회의 내용

- 앞으로 해야할 일 정리.
- 서로가 해온 작업물들 피드백.
- 서버 동기화 관련 내용

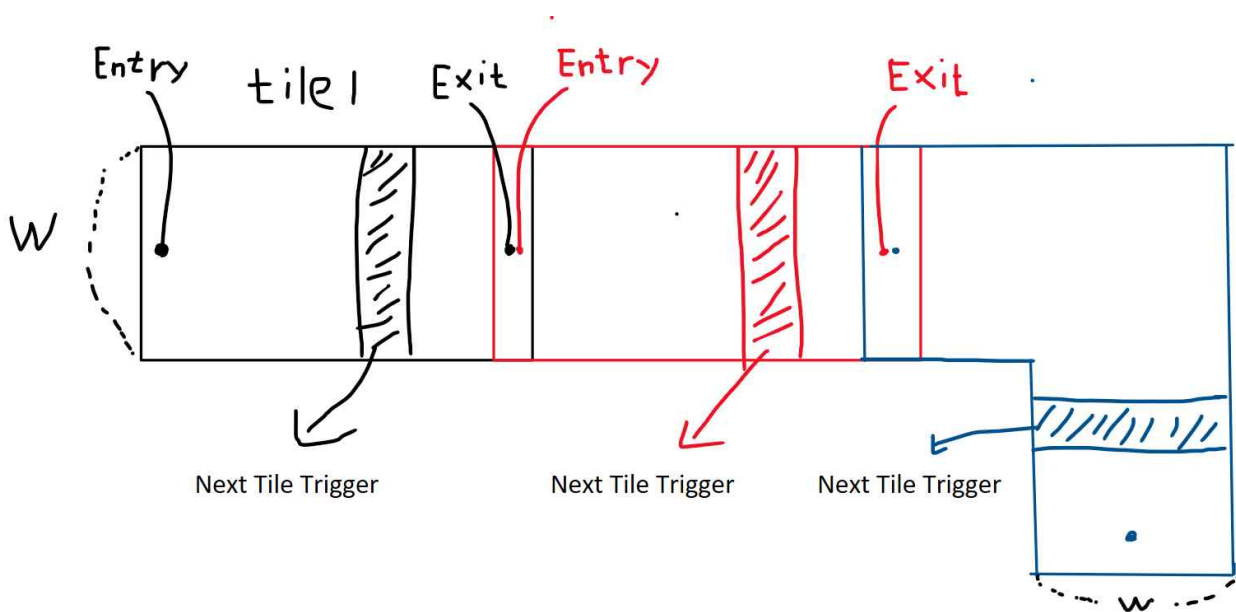
이번 주 한 일

4.27~29 동적으로 무한 맵 생성

기본적인 무한 맵에 대한 설계는 다음과 같다.

타일마다 Entry Point, Next Tile Trigger, Exit Point를 가진 TileMarker가 존재한다.

트럭이 Next Tile Trigger 부분에 도착을 하면,
현재 타일의 Exit 부분에 다음 타일의 Entry 타일이 생성되는 형식이다.



TileMarker는 다음과 같이 만들어주었다.

우선 생성자에서

Entry Arrow, Next Tile Trigger, Exit Arrow를 생성해 주었다.

```
SceneRoot = CreateDefaultSubobject<USceneComponent>(TEXT("SceneRoot"));
SetRootComponent(SceneRoot);

EntryArrow = CreateDefaultSubobject<UArrowComponent>(TEXT("EntryArrow"));
EntryArrow->SetupAttachment(SceneRoot);
EntryArrow->ArrowColor = FColor::Green;

ExitArrow = CreateDefaultSubobject<UArrowComponent>(TEXT("ExitArrow"));
ExitArrow->SetupAttachment(SceneRoot);
ExitArrow->SetRelativeLocation(FVector(5000.0f, 0.0f, 0.0f));
ExitArrow->ArrowColor = FColor::Yellow;

NextTileTrigger = CreateDefaultSubobject<UBoxComponent>(TEXT("NextTileTrigger"));
```

이번 주 한 일

그런 후 BeginPlay()에서 NextTileTrigger에 트럭이 들어오면 HandleNextTileTriggerBeginOverlap() 함수가 호출되도록 이벤트를 바인딩 해주었다.

```
void AStage2TileMarker::BeginPlay()
{
    Super::BeginPlay();

    if (NextTileTrigger)
    {
        NextTileTrigger->SetGenerateOverlapEvents(true);
        NextTileTrigger->OnComponentBeginOverlap.AddDynamic(this, &AStage2TileMarker::HandleNextTileTriggerBeginOverlap);
    }
}
```

HandleNextTileTriggerBeginOverlap() 함수의 내용이다.

```
void AStage2TileMarker::HandleNextTileTriggerBeginOverlap(
    UPrimitiveComponent* OverlappedComponent,
    AActor* OtherActor,
    UPrimitiveComponent* OtherComp,
    int32 OtherBodyIndex,
    bool bFromSweep,
    const FHitResult& SweepResult)
{
    // Engine overlap signature 때문에 매개변수 많음 OtherActor만 사용이 된다.

    if (!OtherActor)
    {
        return;
    }

    ATruck* TriggerTruck = Cast<ATruck>(OtherActor);
    if (!TriggerTruck)
    {
        return;
    }

    if (bHasTriggeredNextTile)
    {
        return;
    }

    bHasTriggeredNextTile = true;
    OnNextTileTriggerEntered.Broadcast(this, TriggerTruck);
}
```

충돌한 액터가 트럭이 아닌 경우 무시를 하고,
외부에 이벤트를 보내준다.

이제 이 이벤트를 TileManager가 받게 될 거다.

이번 주 한 일

TileManager의 BeginPlay에서 StartGeneration 함수를 호출해준다.
StartGeneration함수는 시작 타일을 생성해주는 함수이다.

```
void AStage2TileManager::StartGeneration()
{
    if (bGenerationStarted)
    {
        return;
    }

    ResetGenerationState();
    bGenerationStarted = true;
    // 첫 타일 생성 위치를 Manger의 위치로
    NextSpawnTransform = GetActorTransform();

    // Deterministic Seed 사용: 매번 같은 타일 순서 생성
    if (bUseDeterministicSeed)
    {
        RandomStream.Initialize(RandomSeed);
    }
    else
    {
        RandomStream.GenerateNewSeed();
    }
    // 첫 타일을 생성
    SpawnNextTile();
}
```

타일의 타입은 다음과 같이,
직진, 우회전,좌회전, 시작, 종료타일 이렇게 설정을 하였다.

```
UENUM(BlueprintType)
enum class EStage2TileType : uint8
{
    Start UMETA(DisplayName = "Start"),
    Straight UMETA(DisplayName = "Straight"),
    Left UMETA(DisplayName = "Left"),
    Right UMETA(DisplayName = "Right"),
    Goal UMETA(DisplayName = "Goal")
};
```

이번 주 한 일

SpawnNextTile 함수는 타일 타입을 고르고,
TrySpawnTileLevel() 함수를 호출한다.

```
//현재 생성된 타일이 없으면 Start 타일을 생성
const bool bNeedsStartTile = ActiveTiles.Num() == 0;
const EStage2TileType NextTileType = bNeedsStartTile ? EStage2TileType::Start : ChooseNextTileType();

// 해당 레벨을 NextSpawnTransform 위치에 스트리밍 로드
const TSoftObjectPtr<UWorld> LevelToSpawn = ChooseLevelForTileType(NextTileType);

if (LevelToSpawn.IsNull())
{
    UE_LOG(LogTemp, Warning, TEXT("Stage2TileManager: No level asset configured for tile type %d"), static_cast<int32>(NextTileType));
    return;
}

// 타일을 스폰
if (!TrySpawnTileLevel(LevelToSpawn, NextTileType, NextSpawnTransform))
{
    UE_LOG(LogTemp, Warning, TEXT("Stage2TileManager: Failed to spawn tile level for type %d"), static_cast<int32>(NextTileType));
}
```

TrySpawnTileLevel() 함수는 타일 레벨 하나를 실제 월드에 스트리밍 로드 요청하고,
로드 관리 목록에 등록하는 함수이다.

우선 타일이 똑바로 깔려야 함으로 위치를 보정해준다.

```
// 다음 타일의 EntryArrow가 원하는 위치에 정확히 오도록 레벨 전체 Transform을 보정
FTransform LevelTransformToApply = SpawnTransform;
// 레벨의 경로를 읽어온다.
const FSoftObjectPath LevelPath = TileLevel.ToSoftObjectPath();
if (const FTransform* CachedEntryLocalTransform = CachedEntryLocalTransforms.Find(LevelPath))
{
    // 보정을 해준다.
    LevelTransformToApply = CachedEntryLocalTransform->Inverse() * SpawnTransform;
}
```

그후 맵을 로드 해준다.

로드를 하는 것은 비동기로 실행이 된다.

```
// 런타임 중에 레벨 인스턴스를 동적으로 로드
ULevelStreamingDynamic* StreamingLevel = ULevelStreamingDynamic::LoadLevelInstanceBySoftObjectPtr(
    this,
    TileLevel,
    LevelTransformToApply,
    bLoadSucceeded);

if (!bLoadSucceeded || !StreamingLevel)
{
    return false;
}
```


이번 주 한 일

로드한 맵의 정보를 ActiveTiles 배열에 저장을 해준다.

```
// Load한 Tile의 정보를 저장한다. 나중에 로드 완료된 Tile을 Finalize할 때 사용

// ActiveTiles 배열에 새 원소를 추가하고, 그 참조를 가져온다
FStage2LoadedTile& LoadedTile = ActiveTiles.AddDefaulted_GetRef();
LoadedTile.SourceLevel = TileLevel;           //어떤 레벨 에셋에서 로드된 타일인지 저장
LoadedTile.StreamingLevel = StreamingLevel;
LoadedTile.TileType = TileType;
LoadedTile.RequestedEntryTransform = SpawnTransform;
LoadedTile.AppliedLevelTransform = LevelTransformToApply;
LoadedTile.bInitialized = false;

if (bVerboseLog)
{
    UE_LOG(LogTemp, Log, TEXT("Stage2TileManager: Requested load for %s | Requested=%s | Applied=%s"),
        *TileLevel.ToSoftObjectPath().ToString(),
        *SpawnTransform.GetLocation().ToString(),
        *LevelTransformToApply.GetLocation().ToString());
}
```

TrySpawnTileLevel() 함수는 타일 레벨을 동적으로 로드하고, 타일의 위치를 보정하고 ActiveTiles에 등록하는 함수이다.

이때 동적으로 로드를 할 때 비동기로 로드가 된다.

그래서 로드 요청 직후에는 타일 내부 Actor나 Marker를 바로 사용할 수 없다.

TryFinalizeLoadedTiles() 함수를 매 틱마다 호출하여, 마커를 저장해 주었다.

```
//현재 로드 중이거나 로드 완료된 모든 타일을 검사
for (int32 TileIndex = 0; TileIndex < ActiveTiles.Num(); ++TileIndex)
{
    FStage2LoadedTile& LoadedTile = ActiveTiles[TileIndex];

    // 이미 초기화된 타일 건너뛰기
    if (LoadedTile.bInitialized || !LoadedTile.StreamingLevel)
    {
        continue;
    }

    //레벨이 로드되고, 월드에 활성화 되었는지
    if (!LoadedTile.StreamingLevel->IsLevelLoaded() ||
        !LoadedTile.StreamingLevel->IsLevelVisible())
    {
        continue;
    }

    //마커
    AStage2TileMarker* TileMarker =
        FindTileMarkerFromStreamingLevel(LoadedTile.StreamingLevel);

    // 마커를 못찾으면 대기한다.
    if (!TileMarker)
    {
        continue;
    }

    // 찾았으면 등록
    LoadedTile.TileMarker = TileMarker;

    // 한 Tick에 하나만 처리
    return;
}
```

이번 주 한 일

로드한 맵의 정보를 ActiveTiles 배열에 저장을 해준다.

```
// Load한 Tile의 정보를 저장한다. 나중에 로드 완료된 Tile을 Finalize할 때 사용

// ActiveTiles 배열에 새 원소를 추가하고, 그 참조를 가져온다
FStage2LoadedTile& LoadedTile = ActiveTiles.AddDefaulted_GetRef();
LoadedTile.SourceLevel = TileLevel;           //어떤 레벨 에셋에서 로드된 타일인지 저장
LoadedTile.StreamingLevel = StreamingLevel;
LoadedTile.TileType = TileType;
LoadedTile.RequestedEntryTransform = SpawnTransform;
LoadedTile.AppliedLevelTransform = LevelTransformToApply;
LoadedTile.bInitialized = false;

if (bVerboseLog)
{
    UE_LOG(LogTemp, Log, TEXT("Stage2TileManager: Requested load for %s | Requested=%s | Applied=%s"),
        *TileLevel.ToSoftObjectPath().ToString(),
        *SpawnTransform.GetLocation().ToString(),
        *LevelTransformToApply.GetLocation().ToString());
}
```

TrySpawnTileLevel() 함수는 타일 레벨을 동적으로 로드하고, 타일의 위치를 보정하고 ActiveTiles에 등록하는 함수이다.

이때 동적으로 로드를 할 때 비동기로 로드가 된다.

그래서 로드 요청 직후에는 타일 내부 Actor나 Marker를 바로 사용할 수 없다.

TryFinalizeLoadedTiles() 함수를 매 틱마다 호출하여, 마커를 저장해 주었다.

```
//현재 로드 중이거나 로드 완료된 모든 타일을 검사
for (int32 TileIndex = 0; TileIndex < ActiveTiles.Num(); ++TileIndex)
{
    FStage2LoadedTile& LoadedTile = ActiveTiles[TileIndex];

    // 이미 초기화된 타일 건너뛰기
    if (LoadedTile.bInitialized || !LoadedTile.StreamingLevel)
    {
        continue;
    }

    //레벨이 로드되고, 월드에 활성화 되었는지
    if (!LoadedTile.StreamingLevel->IsLevelLoaded() ||
        !LoadedTile.StreamingLevel->IsLevelVisible())
    {
        continue;
    }

    //마커
    AStage2TileMarker* TileMarker =
        FindTileMarkerFromStreamingLevel(LoadedTile.StreamingLevel);

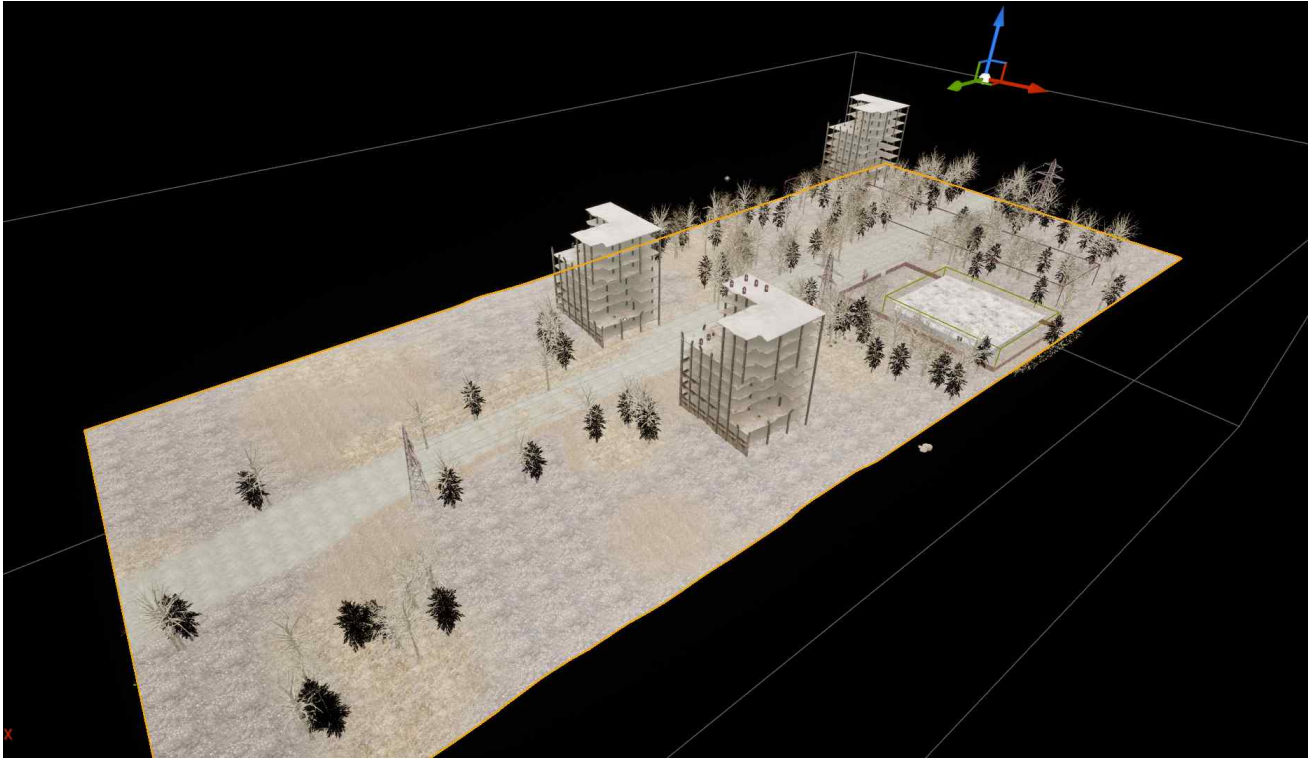
    // 마커를 못찾으면 대기한다.
    if (!TileMarker)
    {
        continue;
    }

    // 찾았으면 등록
    LoadedTile.TileMarker = TileMarker;

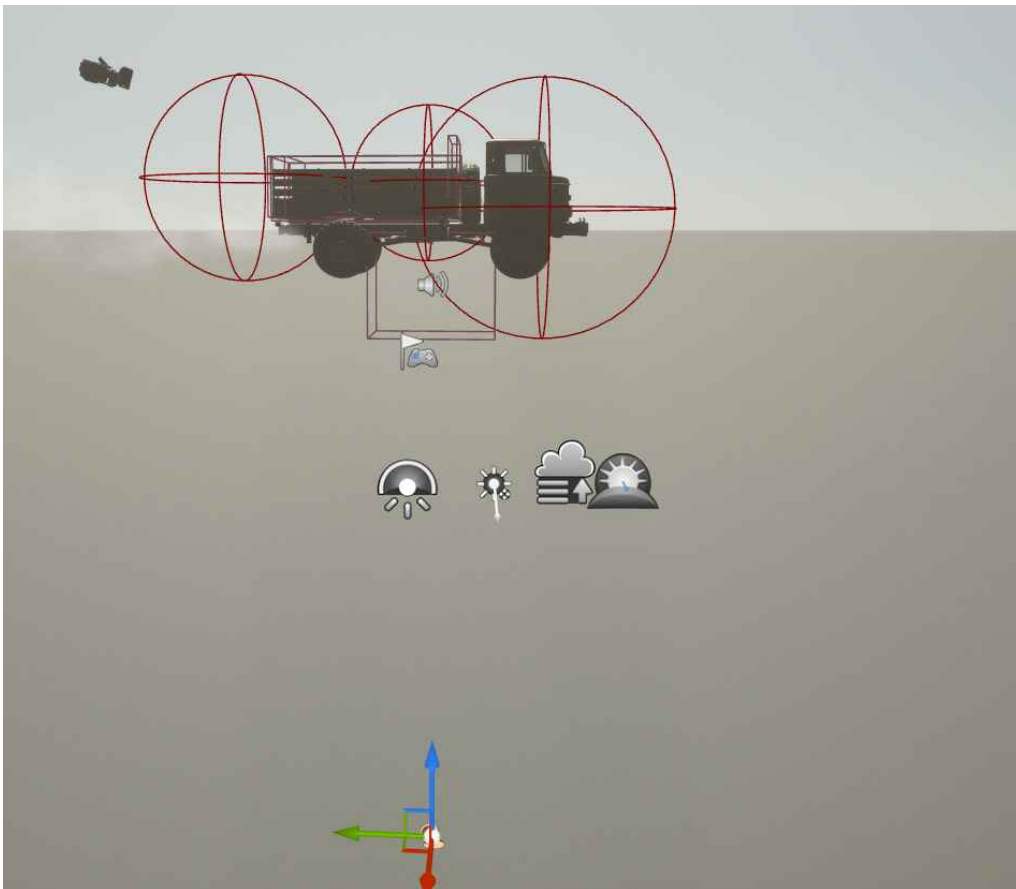
    // 한 Tick에 하나만 처리
    return;
}
```

이번 주 한 일

다음은 직진 타일 맵의 사진이고,
우선 2스테이지에서 직진 타일 맵만 이어가도록 테스트 하였다.



2스테이지 맵에서는 게임 시작전에는 트럭과 플레이어 스폰지점,
타일 매니저만 있는 모습이고



이번 주 한 일

트럭을 타고 계속 달리면 동적으로 맵을 잘 생성하는 것을 볼 수 있다.



사실 저 당시에는 우와 잘 된다 하고 만족을 했지만,

보고서를 정리하면서 문제점이 많다고 느꼈다.

지금 다음맵을 생성하는 트리거를 밟으면, 다음 맵을 생성하며 렉이 좀 걸린다.

아래와 같은 생각들이 들었다.

1. 맵을 트리거를 밟을 때 로드하지말고, 2스테이지 맵 들어갈 때 로드를 하자
2. 현재 특정 타일 이상 지나간 맵은 삭제하는데 풀에 보관 후 재사용하는 게 낫지않나?
3. 풀에 보관하는 방법이 어렵다면, 그냥 뒤에서 앞으로 이동을 시키는 게 낫지않나?

현재 삭제를 하도록 코드를 작성했는데,

삭제 말고 위치를 옮기고, 좀비를 생성하는 방식으로 해보자는 생각이 들었다.

차후에 진행해볼 예정입니다.

이번 주 한 일

4.30 ~ 5.1 맵에 좀비 배치

우선 맵에 좀비를 배치해보았는데,
좀비들이 인지는 하지만, 트럭을 추적 안하는 문제 발생하였다.



원인을 찾아보니, 직진 타일에는 NavMesh를 깔아 두었지만,
2스테이지맵에 깔아두지 않아서 발생한 문제였다.

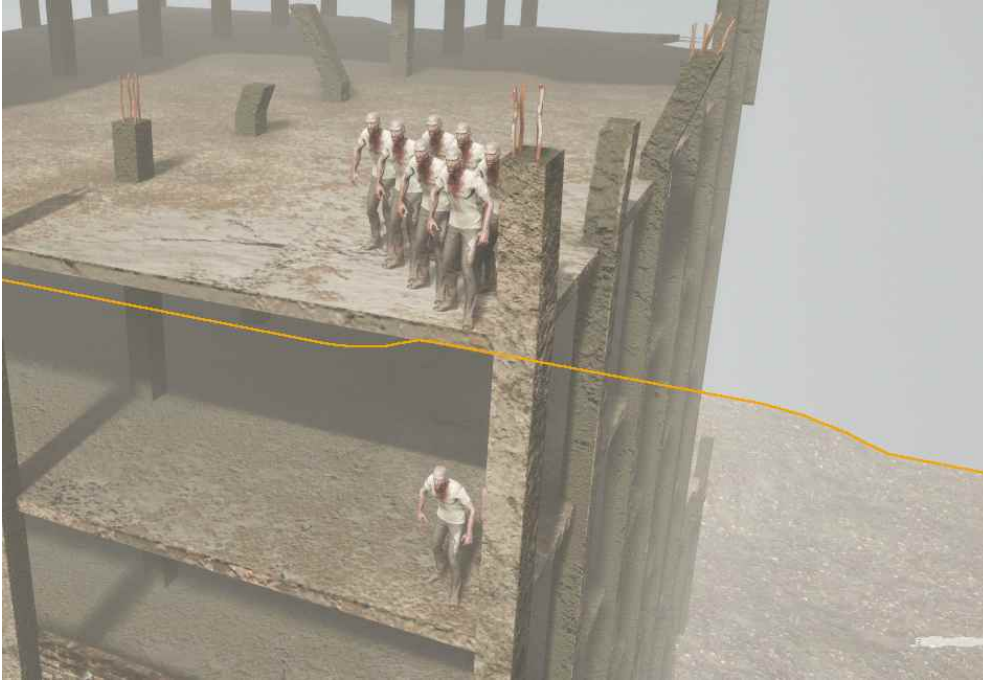
타일맵에는 NavMesh를 지우고, 2스테이지 맵에 크게 두니
좀비들이 잘 쫓아왔다.

아래 사진에서처럼, 좀비들이 차량 소리를 들으면 건물에서 막 뛰어내리고
쏟아져 내리는 것처럼 보이게 만들고 싶다.



이번 주 한 일

그래서 좀비들을 건물에 배치를 시켜놓았는데,

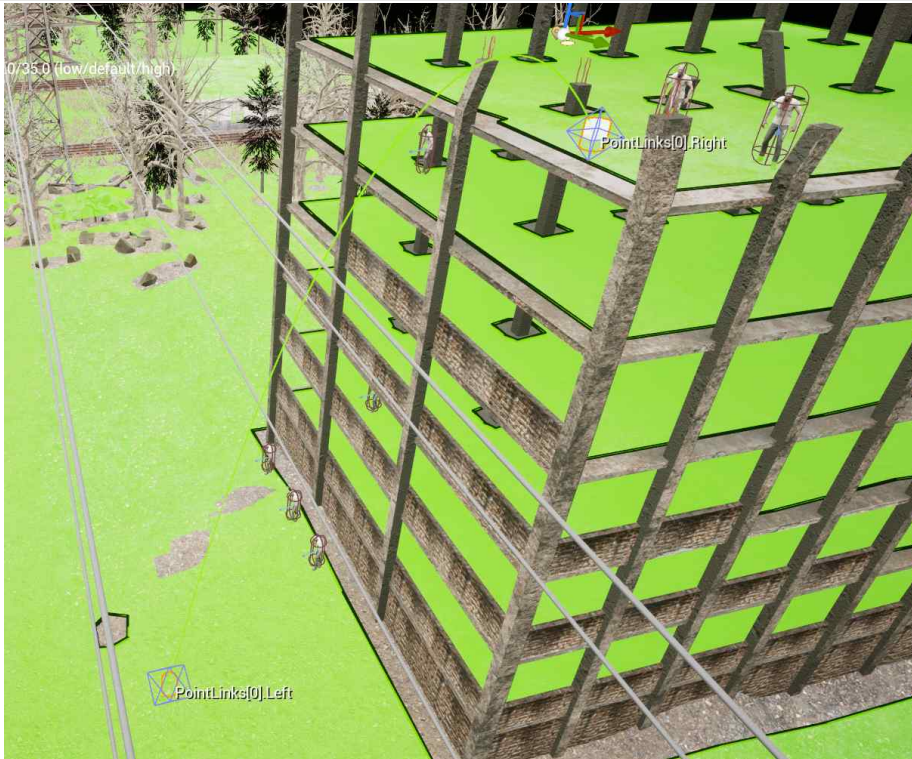


떨어지지 못하고 위에서 어쩔 줄 모르고 있는 문제가 생겼다.

문제는 초록색의 nav mesh가 좀비가 트럭이 있는 곳과 끊겨져 있어 그런 것 같다.



이번 주 한 일



찾아보니, NavLinkProxy를 사용하면 ai가 아래로 떨어지게 할 수 있다 그래서 배치를 해보았다.

하지만 그대로 좀비가 제대로 떨어지지 못하는 모습이라, 건물에 문제가 있나 하고 테스트 맵에서 큐브위에 좀비를 올려놓고 해봐도 여전히 똑같았다.



떨어지지 못하고 어쩔줄 몰라하고있었다.

이번 주 한 일

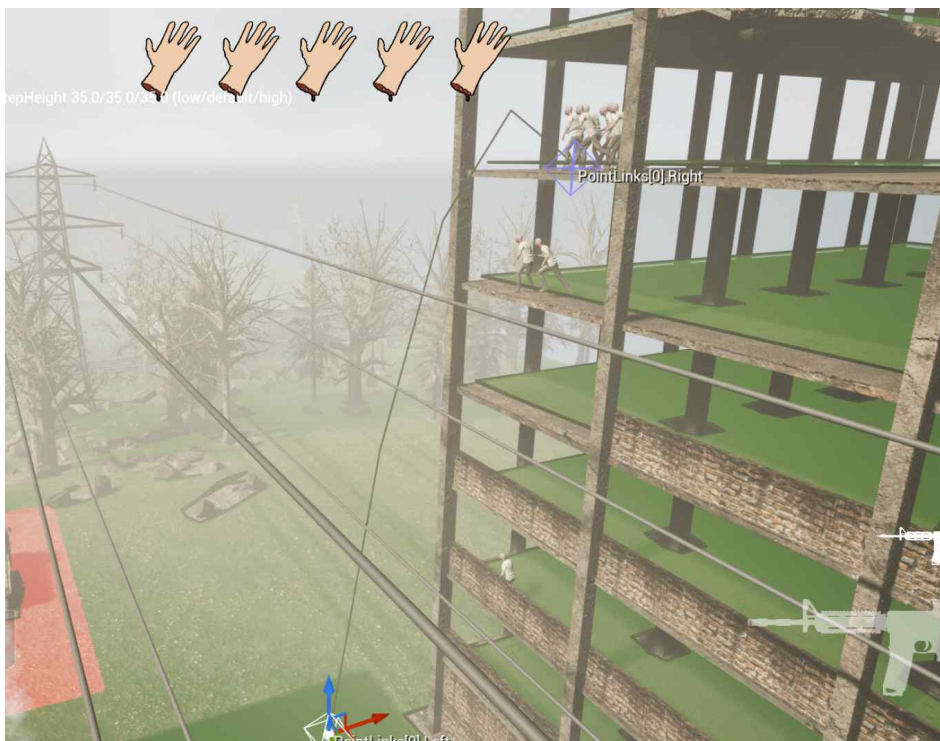
NavLinkProxy를 타기 전에 계속 moveto 함수를 호출하여,
서로 엉키는 문제라고 생각하여,
BTTask_ChaseHuman이 더 이상 바닥 투영 위치로MoveToLocation을 계속 다시 걸지
않고,

기본MoveToActor중심으로 추적하게 바꾸어 주었다.

그러니 이제 테스트맵에서는 좀비가 잘 내려왔다.



하지만 2스테이지 맵에서는 여전히 좀비가 바보같이 못내려오고 있는 모습이다.



5.1 ~ 5.2 보고서 정리

다음 주 할 일

- 트럭 관련 버그 해결
- 주환이랑 트럭 동기화 작업 같이하기
- 1스테이지에서 2스테이지 연결
- 게임 전반적인 완성도 올리기
- 좀비 Ai 강화, 문제해결

특이사항

18주 차 보고서

(2026년 4 월 26 일 ~ 2026년 5 월 2 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

트럭 동기화를 다 마무리 한 줄 알았는데 트럭에서 탑승한 후에 다시 내려지지 않는 것이 확인되어서 급하게 수정을 하게 되었다.

운전자

- 트럭에 탑승 후에 내릴 때 상호작용 키를 서버에 보내주었다.

```
void AFPSBaseCharacter::SyncMovementToServer()  
{  
    if (IsLocallyControlled())  
    {  
        SendMovePacket();  
    }  
}
```

- 하차를 한 후에 캐릭터 동기화가 다시 안되는 문제가 발생해서 서버로 위치를 다시 보내는 SyncMovementToServer() 함수를 만들어주었다.
- 탑승하고 내린 후에 재탑승이 안되어서 다시 재탑승이 가능하게 하였다.

```

switch (pkt.seat_type())
{
case Protocol::TRUCK_SEAT_DRIVER:
    Truck->SetLocallyDriven(bIsLocalPlayer);
    Truck->SetDriverCharacter(Player);
    Player->EnterTruckDriverSeat(Truck);
    if (bIsLocalPlayer && LocalPlayerController)
    {
        LocalPlayerController->Possess(Truck);
        LocalPlayerController->SetControlRotation(Truck->GetActorRotation());
        UE_LOG(LogTemp, Warning,
            TEXT("[TruckDebug] EnterDriver Player=%s Truck=%s Controller=%s ViewTarget=%s TruckLoc=%s\n",
                *GetNameSafe(Player),
                *GetNameSafe(Truck),
                *GetNameSafe(LocalPlayerController),
                *GetNameSafe(LocalPlayerController->GetViewTarget()),
                *Truck->GetActorLocation().ToString(),
                Truck->GetMesh() ? *Truck->GetMesh()->GetComponentLocation().ToString() : TEXT("NoMesh")));
    }
    break;
case Protocol::TRUCK_SEAT_CARGO:
    Player->EnterTruckCargo(Truck);
    break;
case Protocol::TRUCK_SEAT_TURRET:
    if (AMountedMachineGun* MountedWeapon = Truck->GetMountedWeapon())
    {
        Truck->SetMountedWeaponUser(Player);
        Player->EnterMountedWeapon(Truck, MountedWeapon);
    }
    break;
default:
    break;
}

```

HandleEnterTruck, HandleExitTruck에서 로컬 플레이어인지를 더 정확히 판별하도록 수정했다.

```

switch (pkt.seat_type())
{
case Protocol::TRUCK_SEAT_DRIVER:
    Truck->SetLocallyDriven(false);
    if (Truck->GetDriverCharacter() == Player)
    {
        Truck->SetDriverCharacter(nullptr);
    }
    Player->ExitTruckDriverSeat();
    if (blsLocalPlayer && LocalPlayerController)
    {
        LocalPlayerController->Possess(Player);
        LocalPlayerController->SetControlRotation(Player->GetActorRotation());
    }
    Player->SyncMovementToServer();
    break;
case Protocol::TRUCK_SEAT_CARGO:
    Player->ExitTruckCargo();
    if (blsLocalPlayer)
    {
        Player->SyncMovementToServer();
    }
    break;
case Protocol::TRUCK_SEAT_TURRET:
    if (Truck->GetMountedWeaponUser() == Player)
    {
        Truck->SetMountedWeaponUser(nullptr);
    }
    Player->ExitMountedWeapon();
    if (blsLocalPlayer)
    {
        Player->SyncMovementToServer();
    }
}

```

기관총 터렛

- 기관총에 탑승한 후에 내릴 때 트렁크로 내려가져서 바닥으로 내려가지게 했다.
- 하차 후 캐릭터 동기화를 해주었다.
- 기관총 역시 탑승하고 내린 후에 재탑승이 안되어서 다시 재탑승 가능하게 했다.

트렁크

- 트렁크에서 움직일 수 있게 하자고 팀원들과 상의를 해서 트렁크에 올라탄 후에 다시 캐릭터 동기화를 해서 캐릭터 움직임이 다른 클라 화면에 보이게 브로드캐스트 해주었다.
- 트렁크도 마찬가지로 탑승, 재탑승이 가능하게 하였다.

트럭 동기화를 이미 저번주에 끝낸 줄 알았는데 작업을 진행하면 할수록 새로운 문제가 발생하는 것 같다. 다른 작업들을 진행하면서 트럭 동기화에 문제가 발생하면 같이 수정해야될 것 같다.

다음 주 할 일

좀비 동기화, 트럭 동기화 새로운 문제

18주 차 보고서

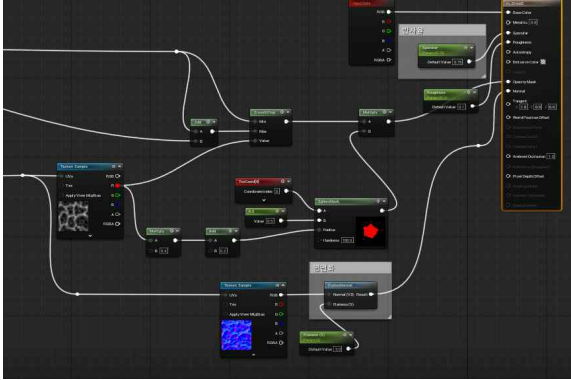
(2026년 4 월 26 일 ~ 2026년 5 월 2 일)

작성자

안윤진

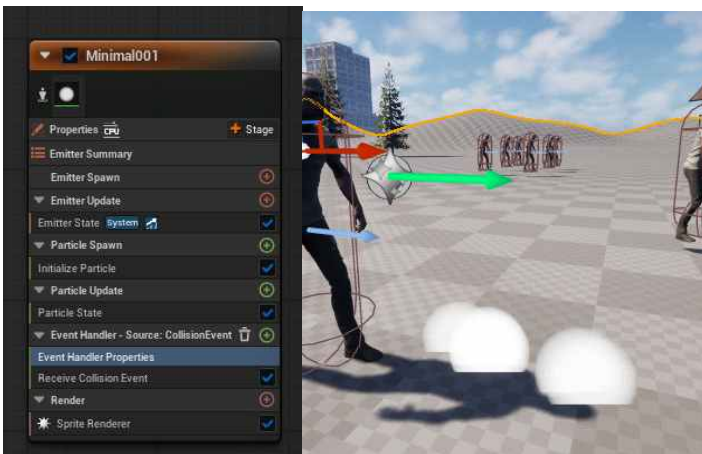
이번 주 한 일

좀 더 과감한 피 튀기는 효과를 찾던 중 괜찮은 자료를 찾아 따라하였다.

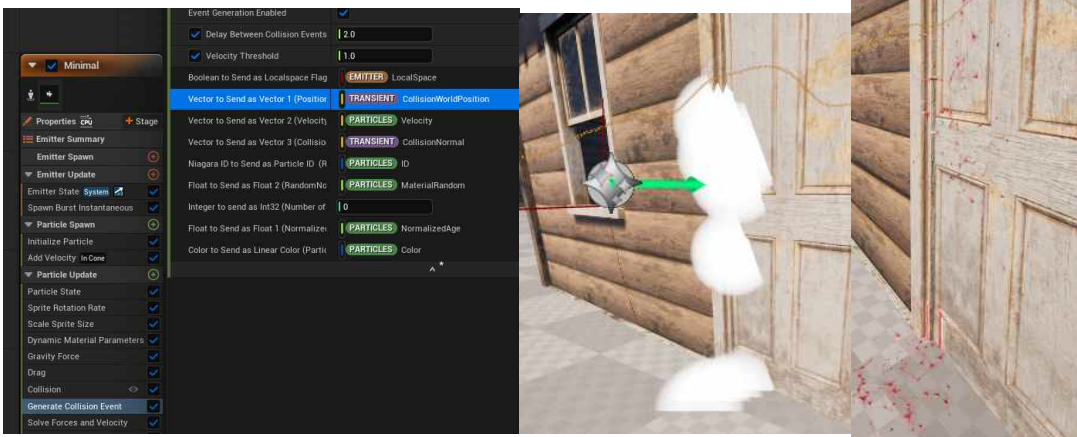


이번 머티리얼 제작에선 노멀텍스처를 사용하였다.

솔직히 머티리얼 제작 부분은 아직 제대로 이해하지 못했다.



한 나이아가라 시스템에 여러 개의 이미터 추가가 가능한 것을 처음 알았다. 피 튀기는 모션은 제작해 두었고, 새로운 이미터를 추가해, 피 튀기는 효과 후 부딪히는 표면 부분에 추가 스프라이트 생성하였다.

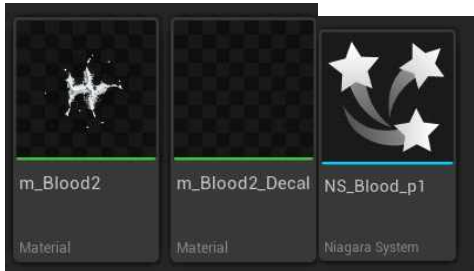


콜리전 부분 월드로 바꿔준다. 벽 등 위치에도 잘 충돌되어 남아있다. 렌더를 스프라이트가 아닌 데칼 렌더러로 변경해주면 저렇게 스티커처럼 남는다. 스프라이트같은 경우에 판 느낌으로 단순한 2D이미지지만, 데칼같은 경우 3D 공간에서 특정 표면에 붙는 이미지로써 실제 메쉬 uv 수정 없이 스티커 붙이는 것처럼 사용 가능하다.

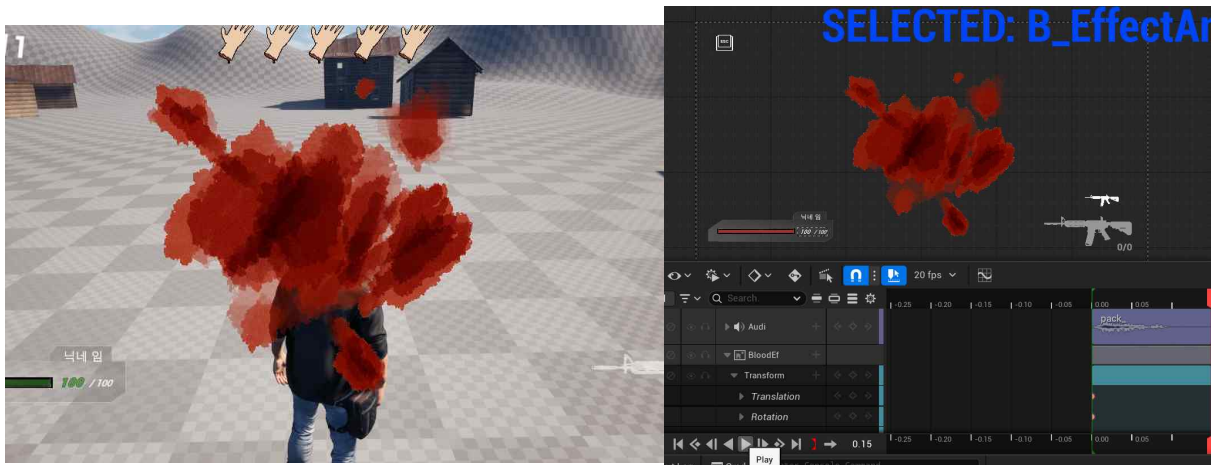
이번 주 한 일



현재 문제점: 데칼 피 색 수정이 안된다. 튀는 피와 그려진 피의 색이 다르다.



이렇게 3개가 추가되었다.



차량으로 좀비를 칠 때 화면에 튀길 피 애니메이션 구현. 팩 소리도 녹음해서 넣어줬다. 이건 좀 한참 후에 차량과 좀비 충돌관련 제대로 할 때 추가하고 수정할 예정.



피 일링이는 효과 테두리 제작. 피가 줄어들면 경고 효과를 주는 용도로 오버워치에서 참고했다. 근데 UMG에서 오퍼시티를 수정하면 반영되어 보여야 하는데 안보이는 문제가 생겼다. 후에 해결하고, 체력과 연동시킬 예정.

다음 주 할 일
피 색 맞추기, 맵2 좀비 떨어질 수 있게 네비게이션 조절, 맵2의 나무 위치 수정

특이사항
아직도 시험기간이라 진도가 별로 나가지 않았다.